

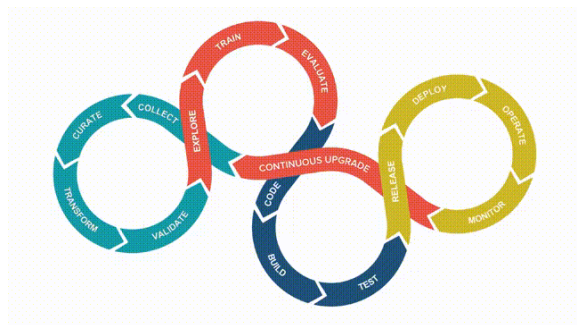


**RV College of
Engineering®**

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world

Machine Learning (MLOps)



Prof. Manasa M
Assistant Professor
Dept of AIML
RVCE



Unit -3

Runtime Environments

- Adaptation from Development to Production Environments
 - Tooling consideration
 - Performance considerations
- Data Access Before Validation and Launch to Production
- Final Thoughts on Runtime Environment

Model Risk Evaluation

- The Origins of ML Model Risk

Quality Assurance for Machine Learning

Key Testing Considerations

Reproducibility and Auditability

Machine Learning Security

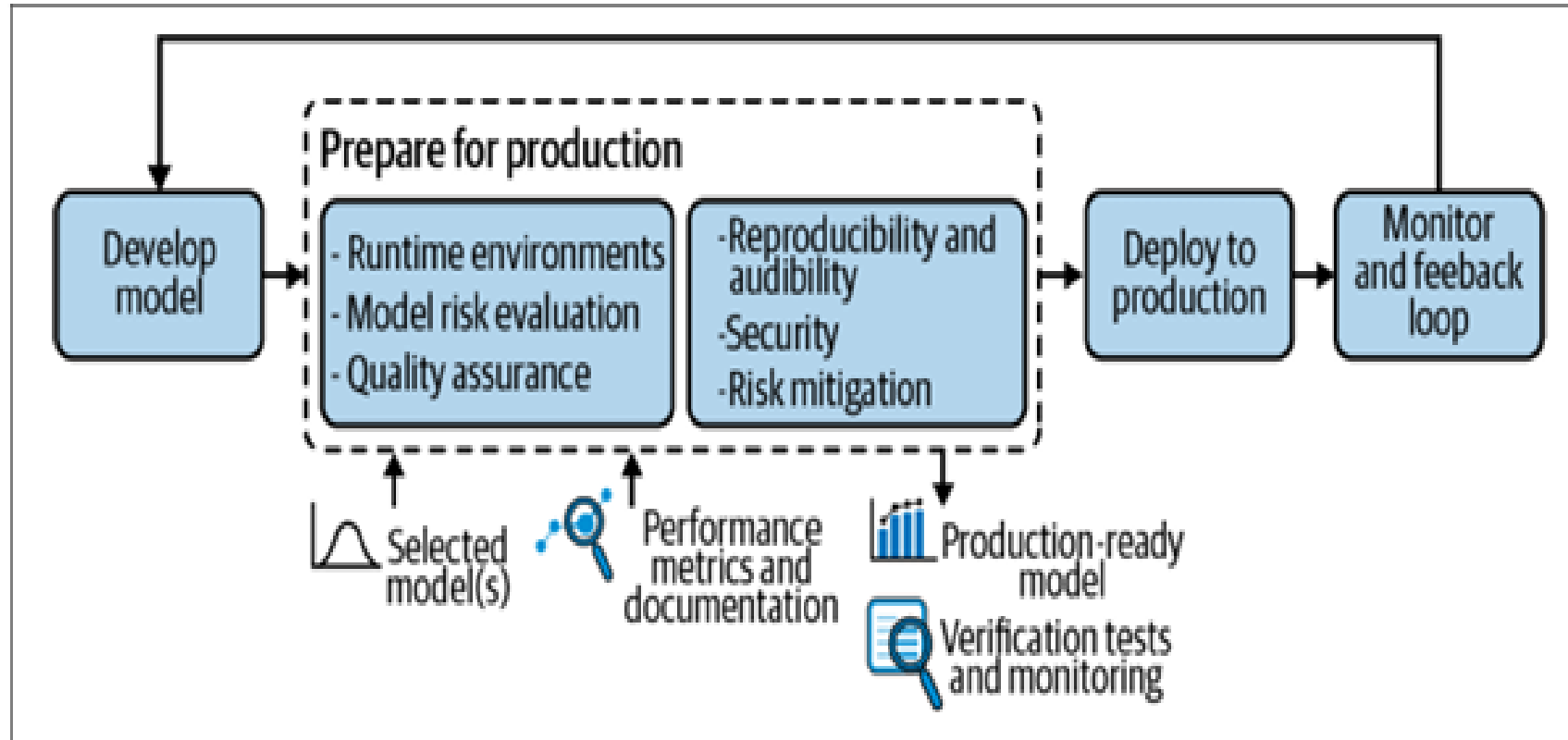
- Adversarial Attack
- Other Vulnerabilities

Model Risk Mitigation

- Interactions Between Model

Unit -3

Steps required to prepare for production





Unit -3

Runtime Environments

- A **runtime environment** refers to the environment in which machine learning models are developed, trained, tested, and deployed, as well as where they are executed during production.
- MLOps combines machine learning, DevOps, and data engineering practices to streamline the development, deployment, and monitoring of machine learning models at scale.
- Production environments take a wide variety of forms:
 - custom-built services,
 - data science platforms,
 - dedicated services like TensorFlow Serving,
 - low-level infrastructure like Kubernetes clusters,
 - JVMs on embedded systems, etc.
- models running in the development environment would be validated and sent as is to production;
- this minimizes the amount of adaptation work and improves the chances that the model in production will behave as it did in development.



Unit -3

Organizations rarely have a “single type” of production setup. Instead, they may use a mix of:

- **Custom-built internal services**
(e.g., a company’s own ML inference server)
- **Data science platforms**
like AWS SageMaker, Azure ML Studio, or Databricks
- **Dedicated serving tools**
such as TensorFlow Serving, TorchServe
- **Low-level infrastructure**
like Kubernetes (K8s), Docker containers
- **Embedded systems**
like JVMs running inside IoT devices, edge devices, or mobile apps



Unit -3

Runtime Environments

[Dev Environment]

OS- Windows

Framework- PyTorch

Python- 3.10

GPU- NVIDIA

Data stored locally

Trained using Jupyter Notebook

[Production Environment]

OS- Linux/Ubuntu

Deployment Platform- Kubernetes Cluster

Serving Tool- Tensorflow Serving

Python -3.9

No GPU



Unit -3

Adaptation from Development to Production Environments

Case 1 : the development and production platforms are from the same vendor

Case 2 : Different vendors

Tooling Considerations – Same Tool Vs Different Tool



Unit -3

Adaptation from Development to Production Environments

A team uses:

- **Docker in development**
- **Docker in production**

So the environment is identical.

The model trained on the laptop is packaged in a Docker container.

The same container runs in Kubernetes.

No code changes needed.



Unit -3

Worst Case

Dev team builds a model using **PyTorch**.

But the production environment only supports:

- **TensorFlow Serving**
- **ONNX Runtime**

What must be done?

- Another engineering team must **re-implement or convert the model** from PyTorch → TensorFlow or ONNX.
- Custom preprocessing code must be rewritten.
- API endpoints must be recreated.
- New validation must be done.
- This can take **months**.



Unit -3

Performance considerations

- Latency –time delay between the input being provided to a system and the output being returned.
- High latency can negatively affect user experience, especially in applications like live streaming, real-time analytics, and on-device processing
- Need to deploy models in low latency edge devices
- Soln : Apply compression techniques – 3 different techniques

With quantization, the model can be trained using 32-bit floating-point numbers and used for inference at a lower precision so that the model requires less memory and is faster while accuracy is mostly preserved.

With pruning, one simply removes weights (or even entire layers) from the neural network. This is a rather radical approach, but some methods allow for the preservation of accuracy.

With distillation, a smaller “student” network is trained to mimic a bigger, more powerful network. Done appropriately, this can lead to better models (as compared to trying to train the smaller network directly from the data)



Unit -3

Data Access Before Validation and Launch to Production

- Data Access for Model Training
- Data Access for Model Validation
- Data Access Management and Governance
- Pre-Production Data Testing and Evaluation

Examples

- a model evaluating apartment prices may use the average market price in a zip code area; however, the user or the system requesting the scoring will probably not provide this average and would most likely provide simply the zip code, meaning a lookup is necessary to fetch the value of the average.
- Freezing data set ??? – when its too large ???



Unit -3

Model Risk Evaluation

Machine Learning (ML) model validation before production is crucial for ensuring that the model performs as expected, is robust, and operates safely in real-world environments.

- What if the model acts in the worst imaginable way?
- What if a user manages to extract the training data or the internal logic of the model?
- What are the financial, business, legal, safety, and reputational risks?

For high-risk applications, it is essential that the whole team (and in particular the engineers in charge of validation) be fully aware of these risks so that they can design the validation process appropriately and apply the strictness and complexity appropriate for the magnitude of the risks.



Unit -3

Model Risk Evaluation

- machine learning risk management covers model risk management practices that are well established in many industries, such as banking and insurance
- machine learning introduces new types of risks and liabilities, and as data science gets democratized, it involves many new organizations or teams that have no experience with more traditional model risk management



Unit -3

Model Risk Evaluation

ML model risk originates essentially from:

- Bugs, errors in designing, training, or evaluating the model (including data prep)
- Bugs in the runtime framework, bugs in the model post-processing/conversion, or hidden incompatibilities between the model and its runtime
- Low quality of training data
- High difference between production data and training data
- Expected error rates, but with failures that have higher consequences than expected
- Misuse of the model or misinterpretation of its outputs
- Adversarial attacks
- Legal risk originating in particular from copyright infringement or liability for the model output
- Reputational risk due to bias, unethical use of machine learning, etc



Unit -3

Model Risk Evaluation

The probability of materialization of the risk and its magnitude can be amplified by:

- Broad use of the model
- A rapidly changing environment
- Complex interactions between models



Unit -3

Quality Assurance for Machine Learning

- In machine learning projects, quality assurance isn't just about testing the product so that it meets quality standards.
- QA for machine learning does not occur only at the final validation stage; rather, it should accompany all stages of model development.
- Its purpose is to ensure compliance with processes as well as ML and computational performance requirements, with a level of detail that is proportionate to the level of risk.
- QA specialists need to work collaboratively with engineers in iterating the ML models, test data and algorithms.
- The QA team needs to set up the tools and methods for the datasets and data models to be valid and the development process to go as smoothly as possible.



Unit -3

Quality Assurance for Machine Learning

Quality Assurance Engineer and ML Engineer

- Building software quality tools for MLOPs is challenging (Scalability issues)
- Error analysis in machine learning projects - Aggregated statistics about the issues will give you a hint on which ones to prioritize and how to optimize data models.
- Validation of the machine learning model on different devices - When building machine learning models, project teams are often constrained by the resources of the devices on which the inference will take place. This is especially the case when developing models for mobile applications where the inference runs directly on the device.
- Label reviews in machine learning data preparation process



Unit -3

Quality Assurance for Machine Learning

- Robust MLOps practices dictate that performing QA before sending to production is not only about technical validation.
- It is also the occasion to create documentation and validate the model against organizational guidelines.
- In particular, this means the origin of all input datasets, pretrained models, or other assets should be known, as they could be subject to regulations or copyrights



Unit -3

Testing

- Data and Model Validation
- Continuous Integration/Continuous Deployment (CI/CD) – Automate the testing and deployment of ML Models
- Versioning and Reproducibility
- **Performance Testing:** Assess the performance of the model in terms of latency, throughput, and resource utilization. This is crucial for deploying models in production environments
- **Monitoring and Logging:** Continuously monitor the performance of the model in production. This includes tracking metrics such as model accuracy, data drift, and system performance
- **Security Testing:** Identify and mitigate potential security vulnerabilities in the ML pipeline. This includes ensuring data privacy and protecting against adversarial attacks
- **Bias and Fairness Testing:** Evaluate the model for biases and ensure that it treats all groups fairly. This is important for maintaining ethical standards and regulatory compliance
- **Scalability Testing:** Test the model's ability to handle large volumes of data and high traffic loads. This ensures that the model can scale effectively in production



Unit -3

Reproducibility and Auditability

- Reproducibility in MLOps also involves the ability to easily rerun the exact same experiment.
- Reproducibility is essential to prove model findings, but also to debug or build on a previous experiment.
- It implies that the model comes with detailed documentation, the data used for training and testing, and with an artifact that bundles the implementation of the model plus the full specification of the environment it was run in
 - **Code Versioning:** Utilize version control systems like GitHub and GitLab to track changes in code. This helps in maintaining a history of code versions used for training and deployment



Unit -3

Reproducibility and Auditability

- Auditability can be an obligation in some highly regulated applications, but it has benefits for all organizations because it can facilitate model debugging, continuous improvement, and keeping track of actions and responsibilities
- Auditability in MLOps is essential for ensuring transparency, accountability, and compliance throughout the machine learning lifecycle.
- Even when MLOps frameworks allow data scientists (or others) to find a model with all its metadata, understanding the model itself can still be challenging



Unit -3

Machine Learning Security

- a deployed model running in its serving framework can present multiple security issues that range from low-level glitches to social engineering.
- Machine learning introduces a new range of potential threats where an attacker provides malicious data designed to cause the model to make a mistake.

Adversarial Attacks

- Adversarial attacks in machine learning involve manipulating input data to deceive models into making incorrect predictions.
- These attacks can pose significant risks, especially in critical applications like finance, healthcare, and autonomous systems.
- **an adversarial stop sign that an autonomous vehicle misinterprets as a yield sign**
- Poisoning Attacks: In this type of attack, malicious data is injected into the training set, causing the model to learn incorrect patterns. This can lead to the model making wrong predictions during deployment



Unit -3

Adversarial Attacks

- Model Inversion Attacks: Attackers use model outputs to infer sensitive information about the training data.
- This can be particularly dangerous if the training data contains private or confidential information
- Membership Inference Attacks: These attacks aim to determine whether a specific data point was part of the training set.
- This can be used to extract sensitive information about individuals in the dataset
- GAN-based Attacks: Generative Adversarial Networks (GANs) can be used to create realistic but fake data to deceive models.
- Adversarial attacks don't necessarily happen at inference time.
- If an attacker can get access to the training data, even partially, then they get control over the system.
- This kind of attack is traditionally known as a poisoning attack in computer security.

Examples

small stickers glued to road signs

Twitter chatbot attack



Unit -3

Adversarial Attacks

- Model Inversion Attacks: Attackers use model outputs to infer sensitive information about the training data.
- This can be particularly dangerous if the training data contains private or confidential information
- Membership Inference Attacks: These attacks aim to determine whether a specific data point was part of the training set.
- This can be used to extract sensitive information about individuals in the dataset
- GAN-based Attacks: Generative Adversarial Networks (GANs) can be used to create realistic but fake data to deceive models.
- Adversarial attacks don't necessarily happen at inference time.
- If an attacker can get access to the training data, even partially, then they get control over the system.
- This kind of attack is traditionally known as a poisoning attack in computer security.

Examples

small stickers glued to road signs

Twitter chatbot attack